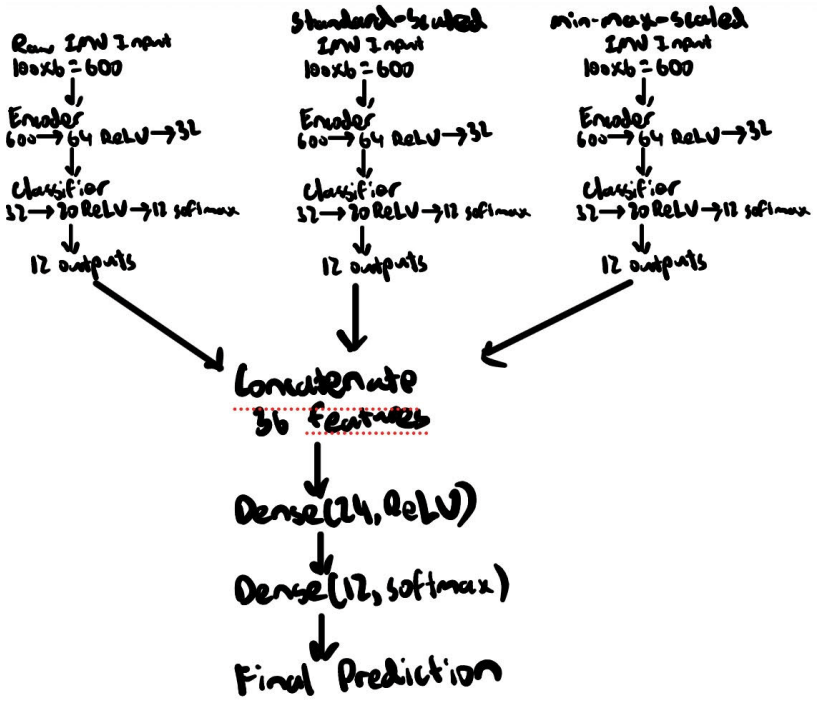
Part I:

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 56 ms, inference 3 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 56 ms, inference 3 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.914062
  lift: 0.085937
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 51 ms, inference 3 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 51 ms, inference 8 ms, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 51 ms, inference 3 ms, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Edge Impulse: https://studio.edgeimpulse.com/public/1013704/live

Part II:

1. 

2.
    a. How pruning is applied and what sparsity schedule is used.
        i. Pruning helps reduce model redundancy and makes the final model smaller for deployment. The magnitude-based pruning schedule gradually increases sparsity instead of removing many weights all at once.
    b. Why the pruning wrappers are stripped before QAT.
        i. They are needed during pruning training, so the model becomes a normal Keras model with the zero-weight pattern already applied.
    c. How QAT is applied after pruning.
        i. QAT is used to simulate int8 quantization during training, letting the model adjust to quantization effects before final int8 conversion.
    d. Why the notebook uses an additional mask-enforcement step during and after QAT instead of using only a simple pruning-then-QAT pipeline.
        i. During QAT, weights continue to update. Without mask enforcement, some weights that were pruned to zero could become nonzero again. The mask-enforcement step keeps the pruned weights at zero during and after QAT, preserving sparsity.
    e. How the final model is converted to an int8 TFLite model and why representative data is needed.
        i. The final model is converted to a full int8 TFLite model. Representative data is needed so TensorFlow Lite can estimate activation ranges and

choose proper quantization scales. Poor representative data can hurt int8 accuracy.

f. How pruning and int8 quantization help when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense.
    i. Pruning reduces the number of active weights, and int8 quantization stores values using smaller integer formats instead of float values. This helps reduce memory use and makes deployment more practical on resource-constrained TinyML hardware.

3. Screenshot:

a.
```
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0195, 0.9531, 0.0000, 0.0195, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039
Standard-scaled model scores: 0.9570, 0.0000, 0.0039, 0.0000, 0.0000, 0.0352, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.5547, 0.0000, 0.1797, 0.2383, 0.0078, 0.0117, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.5039, 0.1719, 0.2031, 0.0039, 0.0039, 0.0234, 0.0000, 0.0039, 0.0430, 0.0391, 0.0039, 0.0000
----------------------------------------
Final prediction: Standing still
Class index: 0
Confidence score: 0.5039
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0195, 0.9492, 0.0000, 0.0234, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039
Standard-scaled model scores: 0.9570, 0.0000, 0.0039, 0.0000, 0.0000, 0.0352, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.5547, 0.0000, 0.1797, 0.2383, 0.0078, 0.0117, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.4922, 0.1758, 0.2109, 0.0039, 0.0039, 0.0234, 0.0000, 0.0039, 0.0430, 0.0391, 0.0039, 0.0000
----------------------------------------
Final prediction: Standing still
Class index: 0
Confidence score: 0.4922
```

b. Acceptable since the MCU was placed flat on a table, the expected behavior is a stationary activity label. The model predicted Standing still, which matches the resting board condition.

c. The prediction was reasonable, but confidence was only around 0.49-0.50, so the model was not extremely confident. This could be due to differences between the original mHealth training data and the real Arduino setup, such as sensor placement, board orientation, unit/scaling differences, and the fact that the training data came from body-worn sensors rather than a board sitting on a table

d. I would collect calibration/training data directly from the MCU using the same board orientation and placement used during testing. I would also add or strengthen an explicit idle/standing still class so the model can more confidently recognize stationary conditions.

4. I did not see the model get completely stuck on one class. When the board was resting flat, it predicted standing still. When I moved the board around, the

prediction changed as shown in the screenshot below. Some predictions were still unexpected because my Arduino movement did not exactly match the original mHealth training motions. This can happen because the training data came from body-worn sensors, while my MCU was hand-moved. Differences in sensor placement, axis orientation, unit scaling, and motion pattern can cause mismatched predictions.

```
Final prediction: Waist bends forward
Class index: 5
Confidence score: 0.7461
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.9453, 0.0117, 0.0195, 0.0000, 0.0000, 0.0000, 0.0000, 0.0195, 0.0000, 0.0039
Standard-scaled model scores: 0.3828, 0.0000, 0.4570, 0.0703, 0.0273, 0.0000, 0.0039, 0.0586, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.4375, 0.0039, 0.1562, 0.0000, 0.0000, 0.3984, 0.0039, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0117, 0.1523, 0.7812, 0.0039, 0.0000, 0.0078, 0.0000, 0.0000, 0.0117, 0.0273, 0.0000, 0.0000
----------------------------------------
Final prediction: Lying down
Class index: 2
Confidence score: 0.7812
----------------------------------------
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0234, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9570, 0.0117, 0.0078
Standard-scaled model scores: 0.0000, 0.0000, 0.1211, 0.0000, 0.1875, 0.0000, 0.1875, 0.5039, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.5898, 0.0391, 0.3672, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.0156, 0.2344, 0.1211, 0.0000, 0.0625, 0.0117, 0.0039, 0.0937, 0.0430, 0.3711, 0.0312, 0.0117
```